

AI Assisted Coding Assignment - 2

Chilukala Sai Architha(2303A52220) - Batch 37

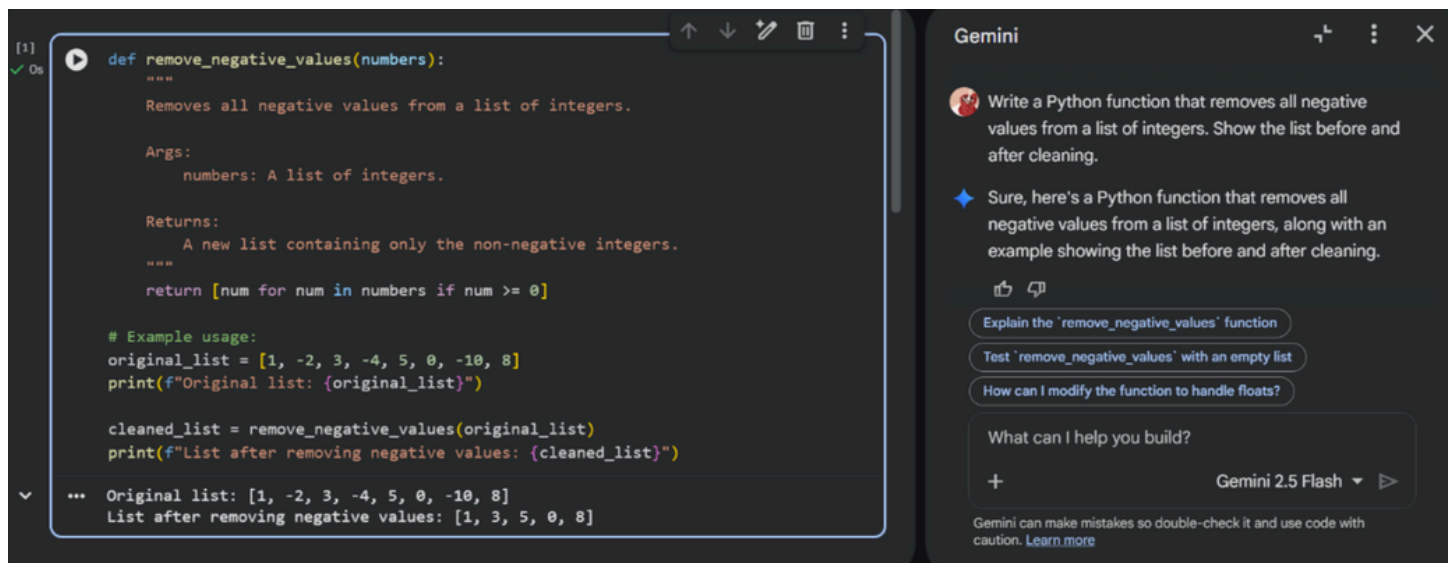
Task 1

Prompt Used: Write a Python function that removes all negative values from a list of integers.

Output:

Before: [10, -3, 5, -8, 12, 0]

After: [10, 5, 12, 0]



The screenshot displays a code editor on the left and the Gemini chat interface on the right. The code editor shows a Python function `remove_negative_values` that filters out negative numbers from a list. The function uses a list comprehension: `return [num for num in numbers if num >= 0]`. Below the function, an example usage is provided, showing the original list `[1, -2, 3, -4, 5, 0, -10, 8]` and the resulting list after removing negative values: `[1, 3, 5, 0, 8]`. The Gemini chat interface on the right shows the prompt: "Write a Python function that removes all negative values from a list of integers. Show the list before and after cleaning." and the response: "Sure, here's a Python function that removes all negative values from a list of integers, along with an example showing the list before and after cleaning." Below the response are several suggested prompts: "Explain the 'remove_negative_values' function", "Test 'remove_negative_values' with an empty list", and "How can I modify the function to handle floats?". At the bottom of the chat interface, there is a text input field with the placeholder "What can I help you build?" and a button labeled "Gemini 2.5 Flash".

```
[1] def remove_negative_values(numbers):  
    """  
    Removes all negative values from a list of integers.  
  
    Args:  
        numbers: A list of integers.  
  
    Returns:  
        A new list containing only the non-negative integers.  
    """  
    return [num for num in numbers if num >= 0]  
  
# Example usage:  
original_list = [1, -2, 3, -4, 5, 0, -10, 8]  
print(f"Original list: {original_list}")  
  
cleaned_list = remove_negative_values(original_list)  
print(f"List after removing negative values: {cleaned_list}")  
  
... Original list: [1, -2, 3, -4, 5, 0, -10, 8]  
List after removing negative values: [1, 3, 5, 0, 8]
```

Gemini

Write a Python function that removes all negative values from a list of integers. Show the list before and after cleaning.

Sure, here's a Python function that removes all negative values from a list of integers, along with an example showing the list before and after cleaning.

Explain the 'remove_negative_values' function

Test 'remove_negative_values' with an empty list

How can I modify the function to handle floats?

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Gemini generated a clean and efficient function using lists. The logic was easy to understand, and it required no need for manual correction.

Task 2

Prompt Used: Create a Python function that counts the number of vowels, consonants, and digits in a string.

Input: Hello123

Output: Vowels: 2, Consonants: 3, Digits: 3

def count_characters(input_string):
 """
 Counts the number of vowels, consonants, and digits in a string.

 Args:
 input_string: The string to analyze.

 Returns:
 A tuple containing (vowel_count, consonant_count, digit_count).
 """
 vowels = "aeiouAEIOU"
 vowel_count = 0
 consonant_count = 0
 digit_count = 0

 for char in input_string:
 if char.isalpha(): # Check if the character is an alphabet
 if char in vowels:
 vowel_count += 1
 else:
 consonant_count += 1
 elif char.isdigit(): # Check if the character is a digit
 digit_count += 1

Gemini

Create a Python function that counts the number of vowels, consonants, and digits in a string. Provide a sample input and output.

◆ Certainly! Here's a Python function that counts the number of vowels, consonants, and digits in a given string, along with a sample usage.

Explain the 'count_characters' function

Test 'remove_negative_values' with an empty list

How can I use 'count_characters' on a list of strings?

What can I help you build?

+

Gemini 2.5 Flash ▾ ▶

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

```
# Get the counts  
v, c, d = count_characters(sample_string)  
  
# Print the results  
print(f"Original string: '{sample_string}'")  
print(f"Number of vowels: {v}")  
print(f"Number of consonants: {c}")  
print(f"Number of digits: {d}")  
  
... Original string: 'Hello World 123!'  
   Number of vowels: 3  
   Number of consonants: 7  
   Number of digits: 3
```

Gemini clearly handled character classification using built-in Python methods like `isalpha()` and `isdigit()` methods.

Task 3

Prompt Used: Write a Python function to check whether a string is a palindrome.

Tool	Code Quality	Readability	Comments
Gemini	Clear and beginner-friendly	High	Explains logic step-by-step
Copilot	Short and optimized	Medium	Assumes prior Python knowledge

```
def is_palindrome(s):  
    """  
    Checks if a given string is a palindrome.  
    A palindrome is a word, phrase, or sequence that reads the same backward as forward,  
    ignoring capitalization and non-alphanumeric characters.  
  
    Args:  
        s: The input string.  
  
    Returns:  
        True if the string is a palindrome, False otherwise.  
    """  
    # Convert to lowercase and remove non-alphanumeric characters  
    cleaned_string = ''.join(char.lower() for char in s if char.isalnum())  
  
    # Compare the cleaned string with its reverse  
    return cleaned_string == cleaned_string[::-1]  
  
# Sample inputs  
string1 = "madam"  
string2 = "A man, a plan, a canal: Panama"  
string3 = "hello"
```

Write a Python function to check whether a string is a palindrome.

Certainly! Here's a Python function to check if a string is a palindrome, along with a sample usage.

The `is_palindrome` function executed successfully, and the output shows the results for the sample

Explain the logic behind the 'is_palindrome' function

How can I make the 'count_characters' function more efficient

Write a new function to reverse a string

What can I help you build?

+ Gemini 2.5 Flash ▾ ▶

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

```
# Test the function and print results  
print(f'"{string1}" is a palindrome: {is_palindrome(string1)}')  
print(f'"{string2}" is a palindrome: {is_palindrome(string2)}')  
print(f'"{string3}" is a palindrome: {is_palindrome(string3)}')  
print(f'"{string4}" is a palindrome: {is_palindrome(string4)}')
```

```
... 'madam' is a palindrome: True  
'A man, a plan, a canal: Panama' is a palindrome: True  
'hello' is a palindrome: False  
'Racecar' is a palindrome: True
```

Welcome Code2.py

C: > Users > lenovo > Desktop > 3-2 > AIAC > Code2.py > ...

```
1 # Write a Python function to check whether a string is a palindrome.  
2 def is_palindrome(s):  
3     # Remove spaces and convert to lowercase for uniformity  
4     s = s.replace(" ", "").lower()  
5     # Check if the string is equal to its reverse  
6     return s == s[::-1]  
7 # Example usage  
8 print(is_palindrome("A man a plan a canal Panama")) # Output: True  
9
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\lenovo> & C:/Users/lenovo/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/lenovo/Desktop/3-2/AIAC/Code2.py
PS C:\Users\lenovo> & C:/Users/lenovo/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/lenovo/Desktop/3-2/AIAC/Code2.py

Task 4

Prompt Used: Explain the following Python palindrome checking function line by line:

```
def is_palindrome(s):
```

```
s = s.lower()
```

```
return s == s[::-1]
```



Explain the following Python palindrome checking function line by line: `def is_palindrome(s): s = s.lower()`
`return s == s[::-1]`



Certainly! Here's a line-by-line explanation of the Python palindrome checking function:

Test the implemented 'is_palindrome' function with more complex cases

Compare the explained 'is_palindrome' function with the one in the notebook

Explain the 'count_characters' function in the second code cell

What can I help you build?

Python palindrome checking function:

```
def is_palindrome(s):  
    s = s.lower()  
    return s == s[::-1]
```

- `def is_palindrome(s):`
 - This line defines a Python function named `is_palindrome` that takes one argument, `s`, which is expected to be a string.
- `s = s.lower()`

lowercase, we ensure that 'Racecar' and 'racecar' are treated the same.

- `return s == s[::-1]`
 - This is the core of the palindrome check.
 - `s[::-1]` creates a reversed version of the string `s` (which is now all lowercase).

The explanation helped me understand slicing operations and how reversing of a string works. Gemini explanation was clear and it was easy to follow.